

# Understanding the NJ Real Estate Market with Data Science

Jibreel Hameed

8/5/2025



# Outline

---

- Introduction
- Executive Summary
- Methodology
- Results
- Conclusion
- Appendix

# Introduction

---

## Background

The current system (or lack thereof) for real estate agents to analyze their local and distance markets is inefficient and time consuming. To properly represent clients and meet their needs, an agent needs to spend many hours throughout the week tracking recent sales, inventory, home prices and conducting comparable sales analysis. With this information, an agent can thoroughly understand market dynamics, when, where and how a buyer should purchase a property or to sell a home. However, markets change on a weekly and monthly basis. This can be overwhelming for new and even established agents as New Jersey has 4-5 different private MLS services which organize and display their data with different methods. This inefficiency can lead to increased frustration and lack of understanding with one's job, increased inability to keep up with fast paced markets, decreased client satisfaction and loss of revenue (salary).

The goal of this project is intended to achieve multiple objectives:

- Create a system which automatically collects, cleans and stores real estate data on a weekly basis for each accessible MLS
- Understand the relationship between a property's sales price and it's physical and locational characteristics
- Determine if a property will be sold based on sales price and physical and locational characteristics
- Create a dashboard to display market statistics as well as provide the ability to conduct data/market analysis on a state and municipal level
- Obtain a deeper knowledge of the real estate market to not only be more informed, but also be a better asset to buyer and seller clients alike



# Executive Summary

---

- 1.Data Collection and Preparation:** The project began with the collection of relevant datasets from publicly available sources as well as paid data sources. Data cleaning and preprocessing were performed to handle missing values, normalize data, and ensure data quality for analysis. Data transformations and feature engineering were also conducted to optimize the dataset for machine learning models.
- 2.Exploratory Data Analysis (EDA):** A thorough exploratory analysis was performed using SQL, data visualization techniques and statistical methods to identify patterns, relationships, and outliers within the data. This phase helped inform the selection of appropriate modeling techniques and variables for the subsequent predictive modeling phase.
- 3.Modeling and Evaluation:** Regression and classification techniques were applied to the data. The models were evaluated using cross-validation, and various performance metrics (such as accuracy, precision, recall, and F1-score) were used to assess their effectiveness. Hyperparameter tuning was also carried out to optimize model performance.
- 4.Result Interpretation and Recommendations:** After developing and evaluating the models, the results were interpreted to draw conclusions about the data and the problem at hand. The analysis provided actionable insights that addressed the project's main objective. Key recommendations were derived from the findings, which were communicated effectively to stakeholders.

# Executive Summary

---

## Summary of Results:

The analysis provided valuable insights into the problem being studied. The key findings include:

- Data Insights:** The exploratory data analysis revealed significant trends and correlations between various features in the dataset, highlighting factors that most influenced the outcomes.
- Model Performance:** The predictive models demonstrated high accuracy and generalization, with the best regression model achieving an R2 Score of 96.8% and the classification model achieving 84% accuracy.
- Actionable Insights:** Based on the analysis, several actionable insights were identified, including the continuation of acquiring more data and conducting more feature engineering. These findings can be used to inform decision-making or guide future research in the domain.



# Section 1: Methodology



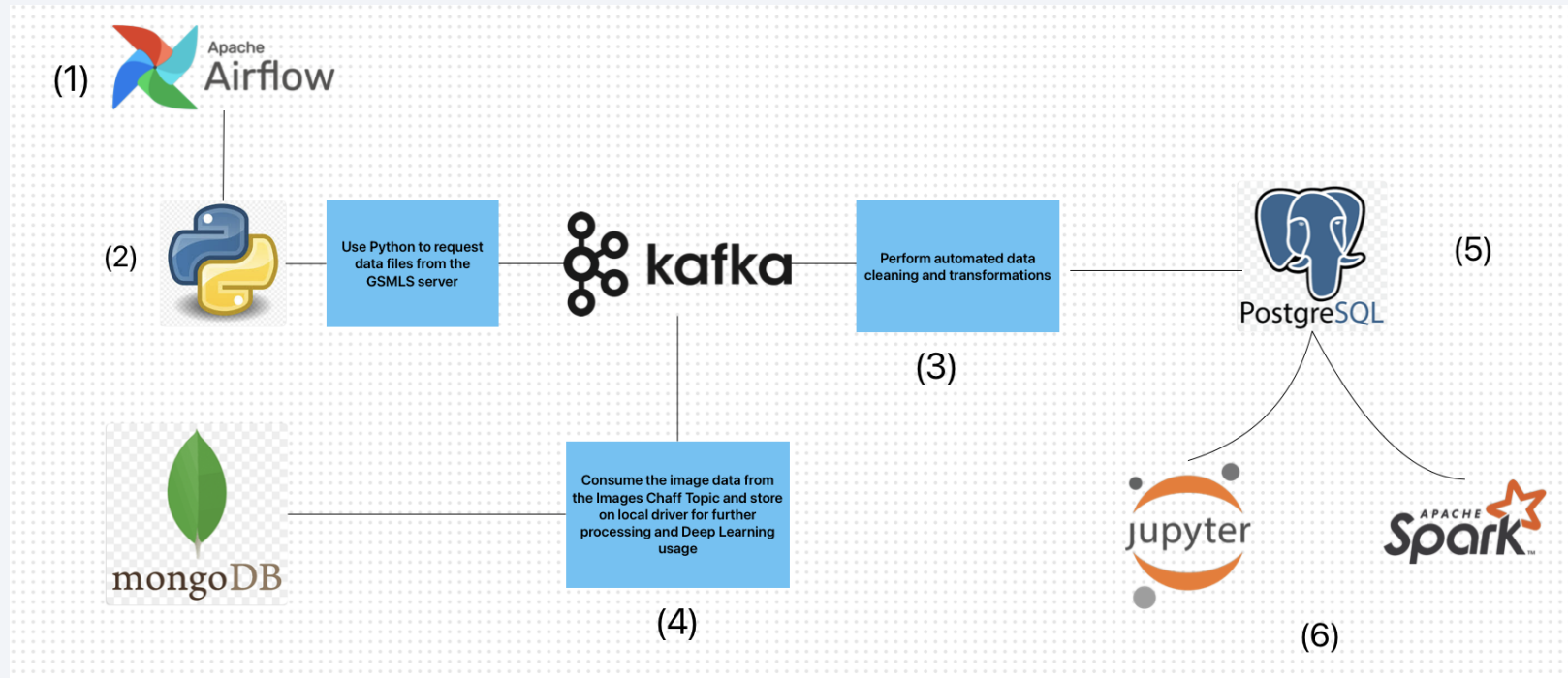
# Methodology

---

## Executive Summary

- Data collection methodology
- Perform data wrangling
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium
- Perform predictive analysis using classification and regression models

# Data Collection



## Pipeline Process:

1. Schedule data scrapes on a weekly basis
2. Use Python and Selenium to automate a script to send data requests from the GSMLS server and produce to Apache Kafka
3. Consume structured data from Apache Kafka to clean and transform
4. Consume image weblinks and residential property characteristics in MongoDB for later consumption
5. Store structured data in PostgreSQL
6. Use Jupyter Notebook for exploratory data analysis and further cleaning for Machine Learning
  - Apache Spark used for more efficient data merging/clean in specific processes



# Data Collection – GSMLS

- Created an automated script using Selenium to receive data in the form of xls/xlsx files
- Files were then encoded as JSON and produced through Apache Kafka to await cleaning/wrangling
- Produced data from 1996 - 2025
- <https://github.com/TheNJineer/GSMLS-Analysis/blob/main/GSMLS.py>

Garden State MLS

AGENT MGMT + SEARCH + ANALYSIS + HOT SHEETS + CONTACTS + FINANCIALS +

Partial Results Custom Result: System Default View:

Map Route ShowingTime Narrow Expand Sort History Stats Save Results Attach Download View/Print Copy MLS #'s Close

| #     | MED | ST           | TOWN | CODE                           | ADDRESS   | LP | C | O | H        | A | T | STYLE | RMS | BRS | BTH       | GAR  | BSMT | SP | SP/LP% | DOM |
|-------|-----|--------------|------|--------------------------------|-----------|----|---|---|----------|---|---|-------|-----|-----|-----------|------|------|----|--------|-----|
| P7    | S   | Roselle Boro | 2914 | 9B Carolyn Ter                 | \$149,900 | H  | A | I | OneFloor | 3 | 1 | 1.0   | 0   | N   | \$145,000 | 97%  | 15   |    |        |     |
| P17 D | S   | Roselle Boro | 2914 | 592 Sherman Ave C2 U: C2       | \$153,000 | H  | A | I | SeeRem   | 3 | 1 | 1.0   | 0   | N   | \$169,000 | 110% | 19   |    |        |     |
| 26    | S   | Roselle Boro | 2914 | 21B Carolyn Ter U: 21b         | \$179,999 | H  | A | I | TwnIntUn | 4 | 2 | 1.0   | 0   | N   | \$205,000 | 114% | 16   |    |        |     |
| 3     | S   | Roselle Boro | 2914 | 41B Carolyn Ter B: 41 U: B     | \$190,000 | H  | A | I | OneFloor | 3 | 1 | 1.0   | 1   | N   | \$200,000 | 105% | 96   |    |        |     |
| D     | S   | Roselle Boro | 2914 | 1909 Wood Ave U: D1            | \$199,000 | H  | A | I | FirstFlr | 5 | 1 | 1.0   | 1   | N   | \$200,000 | 101% | 46   |    |        |     |
|       | S   | Roselle Boro | 2914 | 1601 Wood Ave B: 1601 U: B-1   | \$199,500 | H  | A | I | OneFloor | 3 | 1 | 1.0   | 0   | N   | \$200,000 | 100% | 93   |    |        |     |
|       | S   | Roselle Boro | 2914 | 1619 Wood Ave B: 1619 U: E2    | \$210,000 | H  | A | I | OneFloor | 3 | 1 | 1.0   | 0   | N   | \$218,000 | 104% | 14   |    |        |     |
|       | S   | Roselle Boro | 2914 | 516 Brooklawn Ave U: D2        | \$215,000 | H  | A | I | OneFloor | 5 | 2 | 1.0   | 0   | N   | \$265,000 | 123% | 10   |    |        |     |
|       | S   | Roselle Boro | 2914 | 516 Brooklawn Ave U: D1        | \$215,000 | H  | A | I | OneFloor | 5 | 2 | 1.0   | 0   | N   | \$245,000 | 114% | 9    |    |        |     |
|       | S   | Roselle Boro | 2914 | 519 Brooklawn Ave B: n/a U: H2 | \$219,000 | H  |   | I | OneFloor | 5 | 2 | 1.0   | 0   | N   | \$247,000 | 113% | 31   |    |        |     |
|       | S   | Roselle Boro | 2914 | 1605 Wood Ave B: 1605 U: E2    | \$220,000 | H  | A | I | OneFloor | 5 | 2 | 1.0   | 0   | N   | \$230,000 | 105% | 13   |    |        |     |
|       | S   | Roselle Boro | 2914 | 594 Sherman Ave B: 594 U: F1   | \$220,000 | H  | A | I | OneFloor | 5 | 2 | 1.0   | 0   | N   | \$207,000 | 94%  | 78   |    |        |     |
|       | S   | Roselle Boro | 2914 | 418 E 5th Ave                  | \$284,310 | H  |   | I | Colonial | 6 | 2 | 2.0   | 0   | Y   | \$330,000 | 116% | 53   |    |        |     |
|       | S   | Roselle Boro | 2914 | 327 W 3Rd Ave                  | \$329,800 | H  |   | I | CapeCod  | 6 | 4 | 2.0   | 0   | Y   | \$350,000 | 106% | 3    |    |        |     |
|       | S   | Roselle Boro | 2914 | 105 William St                 | \$329,999 | H  |   | I | CapeCod  | 9 | 4 | 1.1   | 1   | Y   | \$460,000 | 139% | 8    |    |        |     |

MLS

AGENT MGMT + SEARCH + ANALYSIS + HOT SHEETS + CONTACTS + FINANCIALS +

Download Stats 1004 MC Save Search Autoemail Setup Clear All Close

Saved Searches Open House Criteria

Towns

Selected Towns

11 Bergen  
13 Camden  
15 Cumberland  
17 Gloucester  
19 Hunterdon  
21 Middlesex  
23 Morris  
25 Passaic  
27 Somerset  
29 Union  
90 Other

2901- Berkeley Heig  
2904- Elizabeth Cit  
2907- Hillside Twp.  
2910- Mountainside  
2913- Rahway City  
2916- Scotch Plains  
2919- Union Twp.

2902- Clark Twp.  
2905- Fanwood Boro  
2908- Kenilworth Bo  
2911- New Providenc  
2914- Roselle Boro  
2917- Springfield T  
2920- Westfield Town

2903- Cranford Twp.  
2906- Garwood Boro  
2909- Linden City  
2912- Plainfield Ci  
2915- Roselle Park  
2918- Summit City  
2921- Winfield Twp.

Roselle Boro

Enter town code here

# Data Cleaning/Wrangling

---

- Created an automated script which consumed data from Apache Kafka and cleaned the data
- Cleaning process included
  - Removing unwanted characters from features
  - Correcting feature datatypes
  - Parsing column values to create new features
    - Used Regex on columns with string datatypes to identify key features about the property (bank owned, distressed/short sale, interior/exterior features)
  - Removing unwanted columns/features
  - Using tax data from a separate database to correct the livable square feet, lot size and year built features

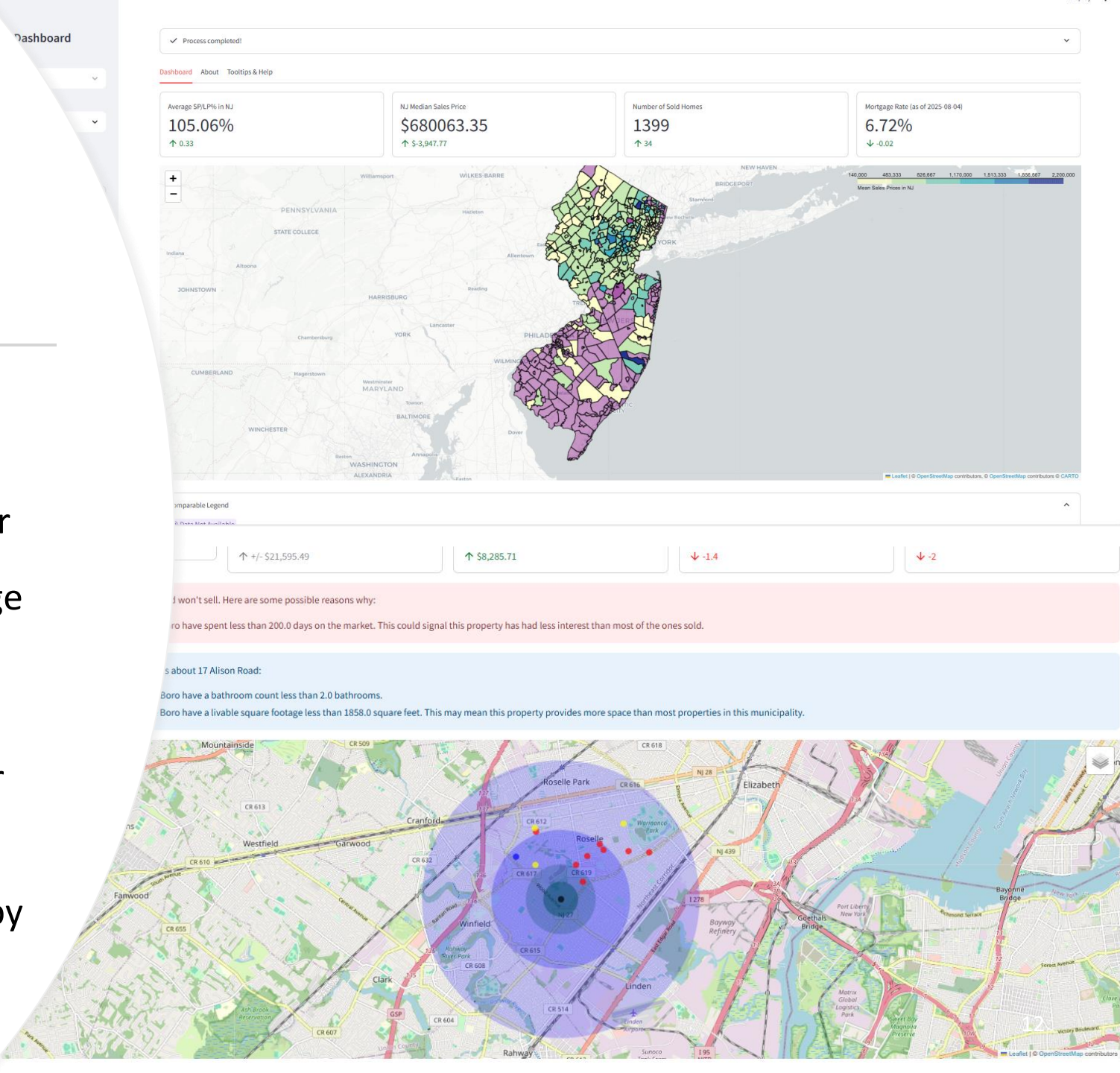
# Data Cleaning/Wrangling (cont'd)

---

- Cleaning processes (cont'd)
  - Use Pandas to create aggregated data tables, per year per municipality to impute data for null and/or incorrect data
  - Use feature skews to understand which data needed transforming to resemble normal data distribution
  - Send associated images to NoSQL database
- Data stored in PostgreSQL once cleaning is complete
- [https://github.com/TheNJineer/GSMLS-Analysis/blob/main/Kafka\\_GSMLSConsumer.py](https://github.com/TheNJineer/GSMLS-Analysis/blob/main/Kafka_GSMLSConsumer.py)

# Build an Interactive Map with Folium

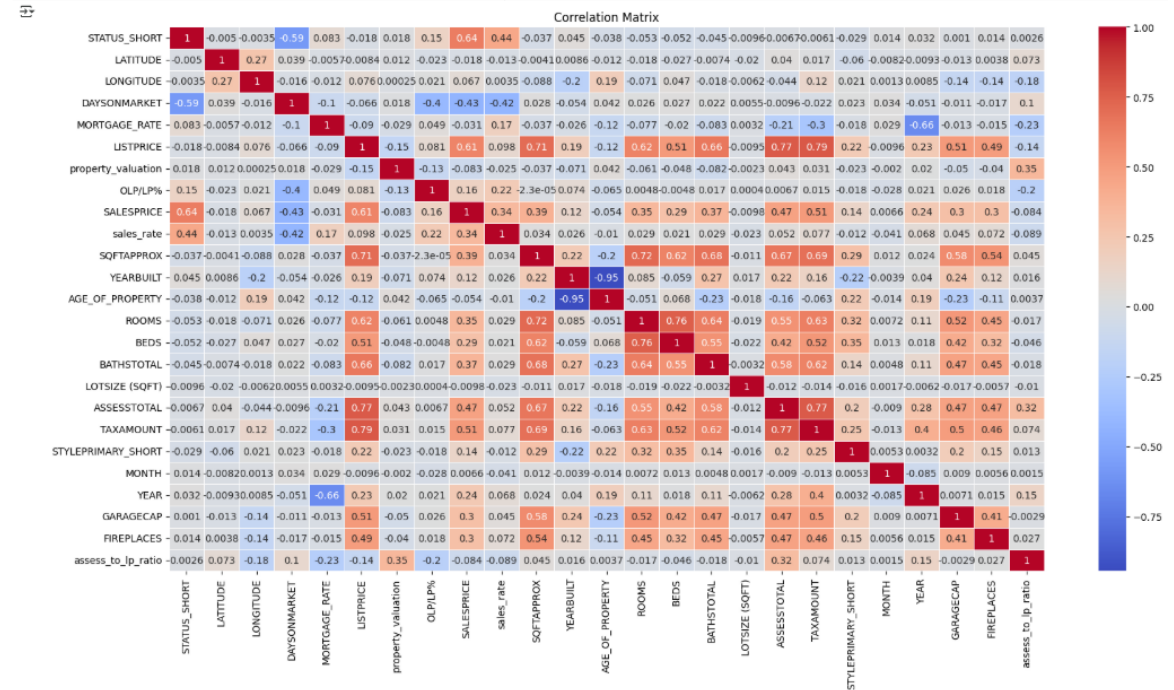
- The dashboard uses Folium to display geospatial data on a state and local level
- The State level view allows users to hover over municipalities to get real estate summaries on median sales price, average listing price percentage received, closed sales and mortgage rates
- The local level view occurs when a user searches the comparable homes for their target property
- [https://github.com/TheNJineer/GSMLS-Analysis/blob/main/GSMLS\\_dashboard.py](https://github.com/TheNJineer/GSMLS-Analysis/blob/main/GSMLS_dashboard.py)





# Linear Correlation Matrix

- The correlation matrix shows how the features were linearly connected
- The Status Short and Sales Price correlations were used as the baseline for minimum input nodes being fed into the neural networks output layers
- Signification correlations (based on magnitude, not direction):
  - List Price – 0.61
  - Tax Amount – 0.51
  - Total Assessment – 0.47
  - Day on Market – 0.43
  - Livable Sqft – 0.39



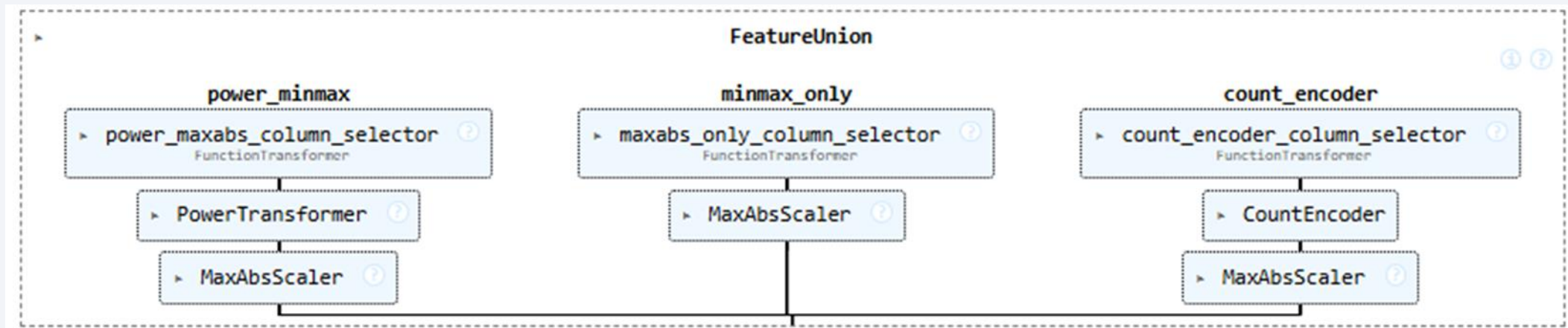
# Predictive Analysis: Transformations

- To prepare the data for machine learning training, I subjected the data to two stages of cleaning
  - Stage 1: Remove the obvious outliers
    - Obvious outliers were ones where I concluded the data couldn't be imputed. An unexhausted list of examples could be if a house was sold and had less than 2 rooms total, was sold but the listing price was \$0, property was sold but have zero bathrooms, or the property spent less than zero days on the market.
  - Stage 2: Create filters using descriptive statistics
    - Looped through the target features to determine the min and max as well as the 25<sup>th</sup>, 50<sup>th</sup> and 75<sup>th</sup> quartiles, respectively. I then standardized the data and filtered the data using the Z-scores so everything was on the same scale
    - To remove outliers but retain as much data as possible, I allowed data up to 5 standard deviations to be used (maximum Z-scores for some features ranged from 11 to 25. I've concluded this could either be incorrect data or data from mansions or unusually sold properties)

```
1 target_col = ['DAYSONMARKET', 'dom_zscore', 'LISTPRICE', 'list_price_zscore', 'list_to_sales_pric
2             'OLP/LP%', 'SALESPRICE', 'ROOMS', 'rooms_zscore', 'SQFTAPPROX', 'sqft_zscore',
3             'LOTSIZE (SQFT)', 'lot_size_zscore', 'BEDS', 'beds_zscore', 'BATHSTOTAL', 'baths_zs
4             'ASSESSTOTAL', 'assessment_zscore', 'TAXAMOUNT', 'tax_zscore',
5             'YEARBUILT', 'year_built_zscore', 'GARAGECAP', 'garages_zscore', 'FIREPLACES', 'fir
6             'sales_rate', 'AGE_OF_PROPERTY', 'MONTH', 'YEAR', 'MORTGAGE_RATE', 'LONGITUDE', 'L
7             'assess_to_lp_ratio', 'assess_to_lp_zscore']
8
9 iqr_dict = {
10     'col': [],
11     'q1': [],
12     'q3': [],
13     'iqr': [],
14     'lower_bound': [],
15     'upper_bound': [],
16     'min': [],
17     'max': [],
18     'skew': []
19 }
20
21 for col in target_col:
22     max_val = gsmls_data[col].max()
23     min_val = gsmls_data[col].min()
24     skew = gsmls_data[col].skew()
25     q1 = float(gsmls_data[col].quantile([0.25]).values[0])
26     q3 = float(gsmls_data[col].quantile([0.75]).values[0])
27     iqr = q3 - q1
28     lower_bound = q1 - (1.5 * iqr)
29     upper_bound = q3 + (1.5 * iqr)
30     iqr_dict['col'].append(col)
31     iqr_dict['q1'].append(q1)
32     iqr_dict['q3'].append(q3)
33     iqr_dict['iqr'].append(iqr)
34     iqr_dict['lower_bound'].append(lower_bound)
35     iqr_dict['upper_bound'].append(upper_bound)
36     iqr_dict['max'].append(max_val)
37     iqr_dict['min'].append(min_val)
38     iqr_dict['skew'].append(skew)
39
40 iqr_df = pd.DataFrame(iqr_dict)
41 iqr_df.set_index('col', inplace=True)
```

# Predictive Analysis: Transformations (cont'd)

- After the data was filtered, the feature skews were recorded to determine if transformations were necessary to resemble normally distributed data
- The data also needed to be standardized, and string features encoded to numerical values
- The following transformations were used:
  - Yeo-Johnson Power Transformation - fix skewed data
  - Min-Max Absolute Scaling – standardization technique
  - Count Encoder – Assign a numerical value to a string value based on frequency of occurrence
- A custom data transformer had to be constructed using Sci-kit Learn Feature Union to ensure the correct transformations were applied to the correct columns



# Predictive Analysis: Model Training

- Before training both classification and regression models, I needed a method to determine the best combination of hyperparameters to use for each model
- TensorFlow Keras doesn't have a native Grid Search CV function, but the closest thing to it is the Keras Tuner object
- Keras Tuner is less exhaustive than Grid Search but it can provide a starting point for good hyperparameters

```
1 # Make dynamic to return model for cls or regression
2
3 def build_model(hp, input_: tuple, batch_size: int, model_type):
4
5     layer_dict = {}
6     input_layer = layers.Input(shape=input_)
7     hp.Fixed('batch_size', batch_size)
8
9     try:
10
11         # Step 1: Decide on if Gradient Clipping will be used
12         if hp.Boolean('Gradient_Clipping') is True:
13             clipnorm_value = hp.Float('clipnorm', min_value=1, max_value=2, step=0.01)
14         else:
15             clipnorm_value = None
16
17         # Step 2: Set the activation choice
18         activation_choice = hp.Choice(name='activation', values=['elu', 'gelu', 'swish'])
19
20         # Step 3: Create the layers for the model
21         if model_type == 'classification':
22
23             # Tune the number of layers for classification in the model
24             cls_layers = hp.Int(name='num_layers_cls', min_value=5, max_value=8)
25
26             for idx, i in enumerate(range(cls_layers)):
27                 dense_layer1 = layers.Dense(
28                     units=hp.Int(name=f'cls_dense_layer {i}', min_value=input_[0], max_value=input_[0]),
29                     activation=activation_choice,
30                     kernel_initializer='he_normal')
31
32                 if idx == 0:
33                     layer_dict[f'cls_dense_layer {idx}'] = dense_layer1(input_layer)
34                 else:
35                     # Inserts the output from the previous layer as the input
36                     layer_dict[f'cls_dense_layer {idx}'] = dense_layer1(layer_dict[f'cls_dense_layer {idx-1}'])
37
38             # Add dropout layers
39             if idx + 1 == cls_layers:
40                 dropout_layer = layers.Dropout(rate=hp.Float(name='cls_dropout_rate', min_value=0.3, max_value=0.8,
41                     layer_dict[f'cls_dropout_layer {idx}'] = dropout_layer(layer_dict[f'cls_dense_layer {idx}'])
42
43             output = layers.Dense(1, activation='sigmoid', name='classification_output')
44
45         elif model_type == 'regression':
```



# Predictive Analysis: Model Training (cont'd)

- An iterative hyperparameter training was created to loop over multiple batch sizes and trials
- This training loop will continue to be utilized as new data is acquired and added to the dataset
- [https://github.com/TheNJineer/GSMLS-Analysis/blob/main/GSMLS\\_DNN\\_Training.py](https://github.com/TheNJineer/GSMLS-Analysis/blob/main/GSMLS_DNN_Training.py)

```
1 base_path = '/content/drive/My Drive/Colab Notebooks/GSMLS'
2
3 best_models_dict = {
4     256: {'Project_Name': 'GSMLS_SimpleCls_DNN_Tuner_256', 'Best_Model': ''},
5     224: {'Project_Name': 'GSMLS_SimpleCls_DNN_Tuner_224', 'Best_Model': ''},
6     192: {'Project_Name': 'GSMLS_SimpleCls_DNN_Tuner_192', 'Best_Model': ''},
7     160: {'Project_Name': 'GSMLS_SimpleCls_DNN_Tuner_160', 'Best_Model': ''}
8 }
9
10 model_type = 'classification'
11 early_stopping_cb = keras.callbacks.EarlyStopping(patience=7, restore_best_weights=True, monitor='val_loss')
12
13 for batch, proj_model in best_models_dict.items():
14     random_search_tuner = kt.RandomSearch(
15         hypermodel = lambda hp: build_model(hp, input_=(23,)), batch_size=batch, model_type=model_type),
16         objective=kt.Objective('val_loss', direction="min"),
17         max_trials=50, executions_per_trial=2, directory='DNN_RandomSearch',
18         project_name=proj_model['Project_Name'], seed=34, overwrite=False)
19
20 # Step 1: Set batch size and set project name
21 if os.path.exists(os.path.join(base_path, 'DNN_RandomSearch', proj_model['Project_Name'])):
22     project_history = random_search_tuner.oracle.get_state()
23     # random_search_tuner.oracle.get_best_trials(num_trials=1)[0].summary()
24     trial_list = list(project_history['display']['trial_number'].values())
25
26     try:
27         if trial_list[-1] == 50:
28             print(f"{proj_model['Project_Name']} has finished tuning. Moving to next configuration")
29             best_models_dict[batch]['Best_Model'] = random_search_tuner.get_best_hyperparameters()[0].values
30             continue
31         else:
32             print(f"{proj_model['Project_Name']} has not finished tuning. Continuing from {trial_list[-1] + 1}")
33
34     except IndexError:
35         pass
36
37 # Step 2: Initiate RandomSearch function
38 if model_type == 'classification':
39     random_search_tuner.search(x_train_transformed, y_train_encoded,
40                               epochs=50, batch_size=batch, sample_weight=sample_weights,
41                               validation_data=(x_val_transformed, y_val_encoded), verbose=1,
42                               callbacks=[early_stopping_cb])
43
44 elif model_type == 'regression':
45     random_search_tuner.search(x_train_transformed, y_true_train,
46                               epochs=50, batch_size=batch,
47                               validation_data=(x_val_transformed, y_true_val), verbose=1,
48                               callbacks=[early_stopping_cb])
49
50 if best_models_dict[160]['Best_Model'] != '':
51     print('DNN Grid Search Completed')
```

# Predictive Analysis: Model Training (cont'd)

- Created a custom evaluation metric for the regression model
  - Because the data contained homes that were both sold and expired on market, the expired homes had sales prices of \$0. This negatively skewed the target data range and therefore the mean squared error metric
  - The custom evaluator filtered out the expired homes and only focused on the ones which sold
- Created functions which dynamically built the neural network architecture to speed up the training process
- [https://github.com/TheNJineer/GSMLS-Analysis/blob/main/GSMLS\\_DNN\\_Training.py](https://github.com/TheNJineer/GSMLS-Analysis/blob/main/GSMLS_DNN_Training.py)

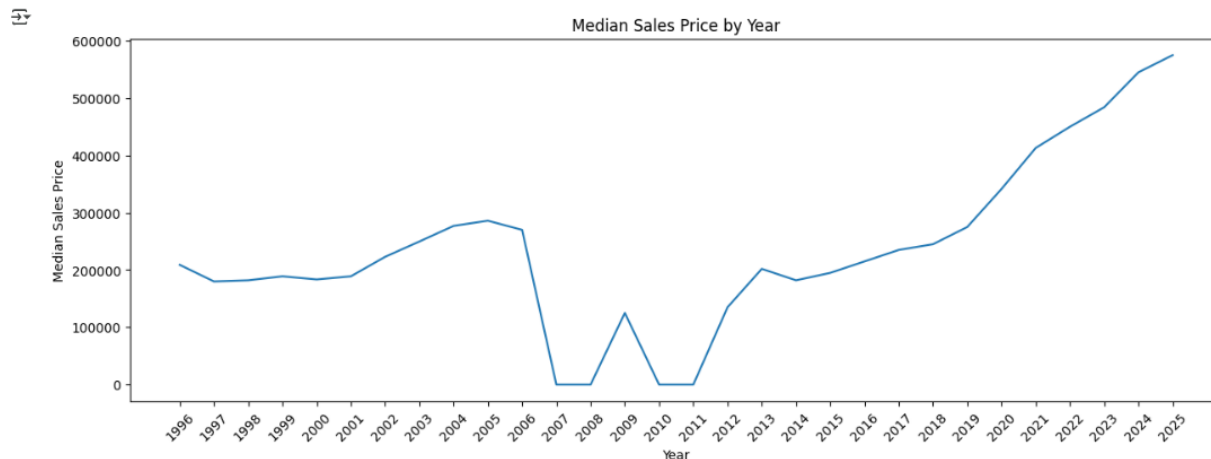
```
1 # Create custom loss function to help evaluate the DNN better
2
3 def masked_mse_metric(y_true, y_pred, sample_weight=None):
4
5     actual_prices = y_true[:, 0]
6     sold_flags = y_true[:, 1]
7
8     # Calculate the squared error
9     square_error = tf.square(actual_prices - tf.squeeze(y_pred))
10
11     # Apply the mask to ignore losses when properties arent sold
12     masked_squared_error = square_error * sold_flags
13
14     # Compute the mean only over the active sold samples (avoid dividing by the full batch size)
15     # Add a small epsilon to avoid dividing by zero
16     total_weight = tf.reduce_sum(sold_flags * (sample_weight if sample_weight is not None else 1.0)) + 1e-6
17     mean_squared_error = tf.reduce_sum(masked_squared_error) / total_weight
18
19     # return masked_squared_error
20     return mean_squared_error
21
```

```
1 def simple_classification(num_layers):
2
3     tf.random.set_seed(34)
4
5     # Create the necessary layers
6
7     model = keras.Sequential()
8     model.add(layers.Input(shape=x_train_transformed.shape[1:]))
9
10    for i in range(num_layers):
11        # Add layers to the model
12        model.add(layers.Dense(23, activation='gelu', kernel_initializer='he_normal'))
13
14        if i == num_layers - 1:
15            model.add(layers.Dropout(0.7))
16            model.add(layers.Dense(1, activation='sigmoid', name='classification_output'))
17
18    optimizer = optimizer = keras.optimizers.Adamax(learning_rate=0.0012324, weight_decay=0.0010279, clipnorm=1.39)
19    model.compile(
20        loss='binary_crossentropy',
21        optimizer=optimizer,
22        metrics=['accuracy'])
23
24    return model
```

An aerial photograph of a suburban neighborhood, showing rows of houses with grey roofs and green lawns, interspersed with trees. The image is partially obscured by a dark overlay on the right side, where the text is located.

## Section 2: Insights Drawn from EDA (SQL Queries and Data Visualization)

# Median Sales Price by Year



```
1 query="""
2 SELECT gsmls_imputed_data.year, percentile_cont(0.5) WITHIN GROUP(ORDER BY CAST(salesprice as numeric)) as median_sales_price
3 FROM gsmls_imputed_data
4 GROUP BY gsmls_imputed_data.year;
5 """
6
7 med_sales_price = pd.read_sql_query(query, tax_engine)
```

```
1 fig, ax = plt.subplots(figsize=(15, 5))
2
3 ax.plot('year', 'median_sales_price', data=med_sales_price)
4 ax.set_title('Median Sales Price by Year')
5 ax.set_xlabel('Year')
6 ax.set_ylabel('Median Sales Price')
7
8 plt.show()
```

The median sales price of homes in New Jersey has steadily increased after the Great Financial Crisis

\*This plot was created using filtered data trained on by machine learning algorithms, therefore certain data can be missing

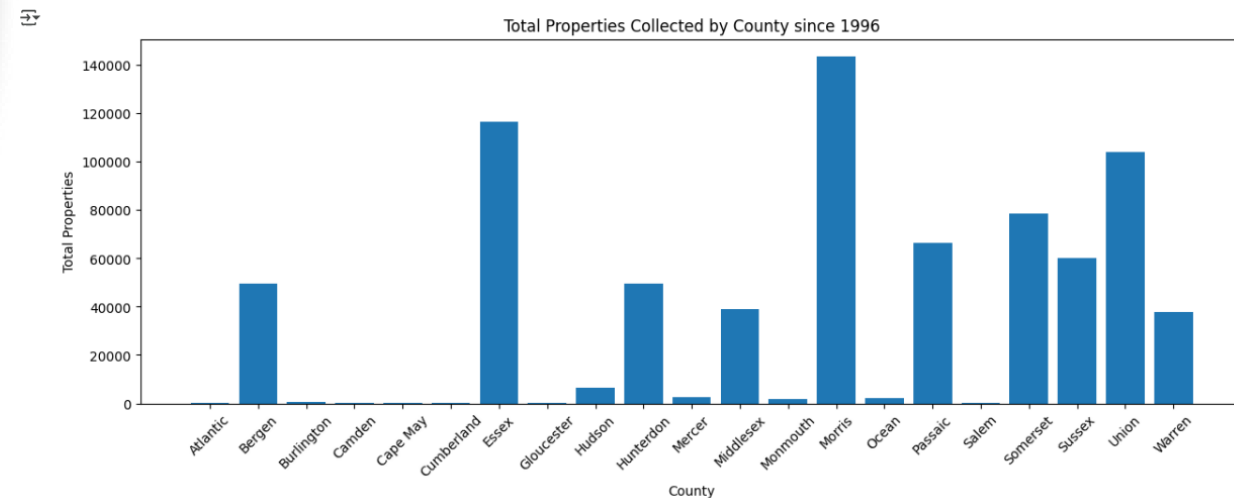


# Most/Least Transactions Collected by County

- Before commencing this project, it was understood that this MLS service covered a large area of NJ but some counties reported to separate/regional MLS
- The bar chart shows that since data was first collected, most of the data received are from counties located in Northern and Central New Jersey
- Morris, Essex and Union counties have reported the most residential real estate transactions respectively since 1996

```
1 query="""
2 SELECT county, COUNT(*) as total_properties
3 FROM gsmls_imputed_data
4 GROUP BY county;
5 """
6
7 total_properties = pd.read_sql_query(query, tax_engine)

] 1 fig, axs = plt.subplots(figsize=(15,5))
2
3 axs.bar('county', 'total_properties', data=total_properties)
4 axs.set_title('Total Properties Collected by County since 1996')
5 axs.set_xlabel('County')
6 plt.xticks(rotation=45)
7 axs.set_ylabel('Total Properties')
8
9 plt.show()
```



```

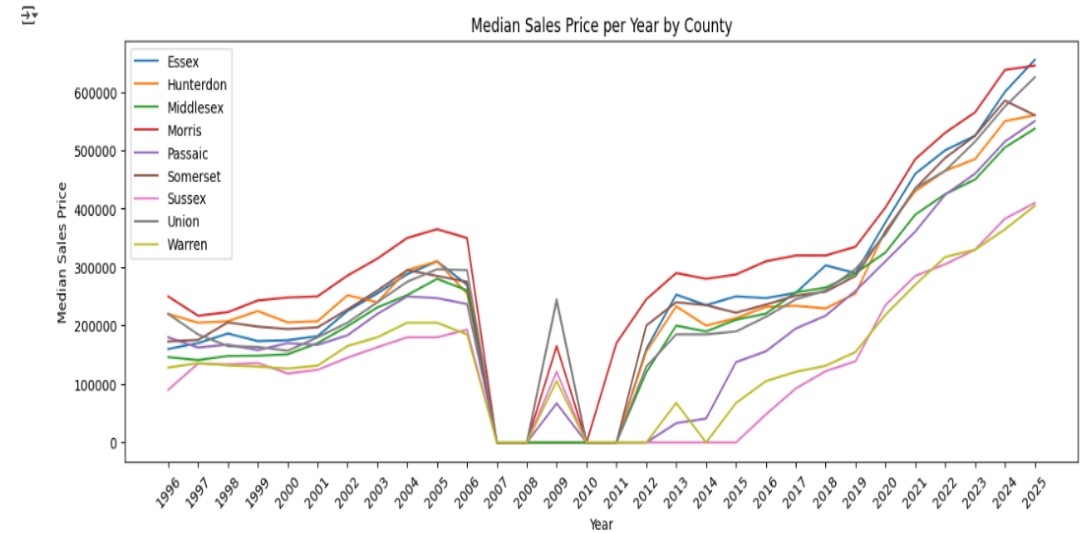
1 query="""
2 SELECT gsmls_imputed_data.year, county, percentile_cont(0.5) WITHIN GROUP(ORDER BY CAST(salesprice as numeric)) as median_sales_price
3 FROM gsmls_imputed_data
4 WHERE county IN ('Essex', 'Morris', 'Middlesex', 'Hunterdon', 'Passaic', 'Somerset', 'Sussex', 'Union', 'Warren')
5 GROUP BY gsmls_imputed_data.year, county;
6 """
7
8 med_sales_price = pd.read_sql_query(query, tax_engine)

```

```

1 fig, axs = plt.subplots(figsize=(15, 5))
2
3 for county in med_sales_price['county'].unique():
4     axs.plot('year', 'median_sales_price', data=med_sales_price[med_sales_price['county'] == county], label=county)
5
6 axs.set_title('Median Sales Price by Year')
7 axs.set_xlabel('Year')
8 axs.set_ylabel('Median Sales Price')
9
10 plt.legend()
11 plt.show()

```



## Median Sales Price by Year By County

- Of the counties with sufficiently reported data in this MLS, the Top 3 counties with the highest median sales price are Essex, Morris and Union, respectively. Somerset County seems to be the only county to have decreased YoY in 2025. However, this will be revisited once we have a full year of sales data

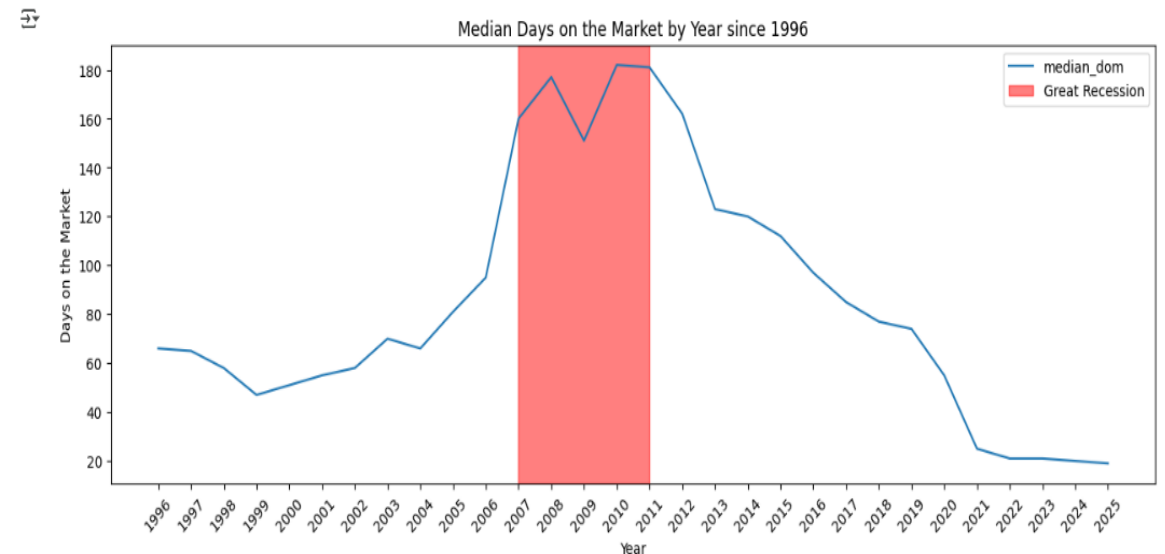
\*This plot was created using filtered data trained on by machine learning algorithms, therefore certain data can be missing

# Median Days on the Market by Year

- Properties in New Jersey spent the most time on the market during the Great Financial Crisis with the peak median Days on Market (DOM) occurring in 2008 and 2010
- Since 2011, a property's time on market before being sold has steadily decreased
- After COVID in 2020, the median DOM for properties sold in NJ has not gone above 30 days

```
1 query="""
2 SELECT gsmls_imputed_data.year, percentile_cont(0.5) WITHIN GROUP(ORDER BY CAST(daysonmarket as numeric)) as median_dom
3 FROM gsmls_imputed_data
4 GROUP BY gsmls_imputed_data.year;
5 """
6
7 med_dom = pd.read_sql_query(query, tax_engine)
```

```
1 fig, ax = plt.subplots(figsize=(15,5))
2
3 ax.plot('year', 'median_dom', data=med_dom)
4 ax.set_title('Median Days on the Market by Year since 1996')
5 ax.set_xlabel('Year')
6 ax.set_ylabel('Days on the Market')
7
8 plt.xticks(range(1996, 2026, 1), rotation=45)
9 plt.axvspan(2007, 2011, color='red', alpha=0.5, label='Great Recession')
10 plt.legend()
11 plt.show()
```

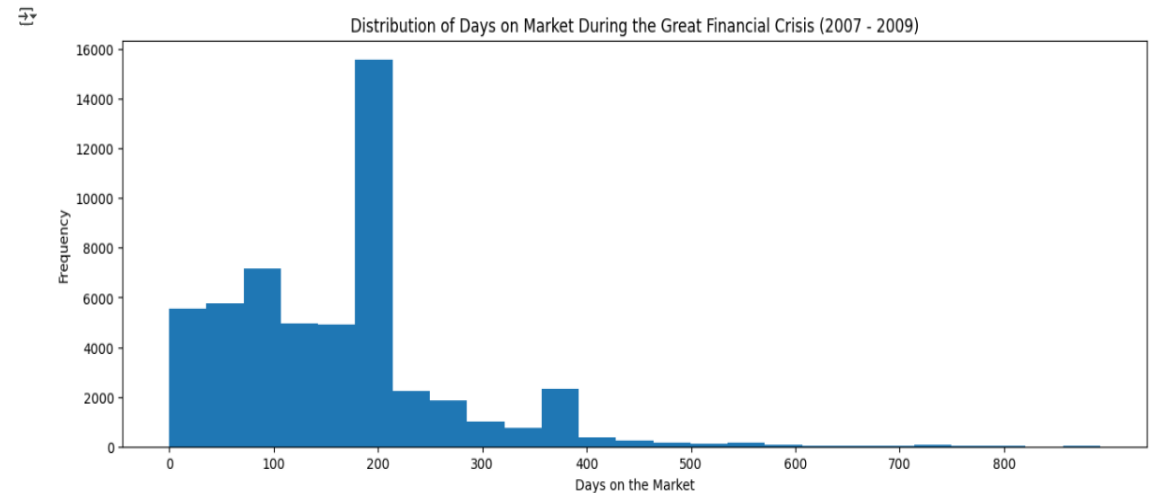


# Days on the Market Distribution (Great Financial Crisis)

- Continuing to focus on the period of the Great Financial Crisis, the graph below shows the distribution of DOM for properties sold or expiring
- The data is right skewed with some properties spending a year or more on the market before being sold or expiring
- The most frequent occurrence was around 200 days

```
1 query="""
2 SELECT daysonmarket
3 FROM gsmls_imputed_data
4 WHERE listdate BETWEEN '2007-10-01' AND '2009-03-31'
5 OR closeddate BETWEEN '2007-10-01' AND '2009-03-31';
6 """
7
8 recession_data = pd.read_sql_query(query, tax_engine)
```

```
1 fig, axs = plt.subplots(figsize=(15,5))
2
3 axs.hist('daysonmarket', data=recession_data, bins=25)
4 axs.set_title('Distribution of Days on Market During the Great Financial Crisis (2007 - 2009)')
5 axs.set_xlabel('Days on the Market')
6 axs.set_ylabel('Frequency')
7
8 plt.xticks(range(0,900, 100))
9 plt.show()
```





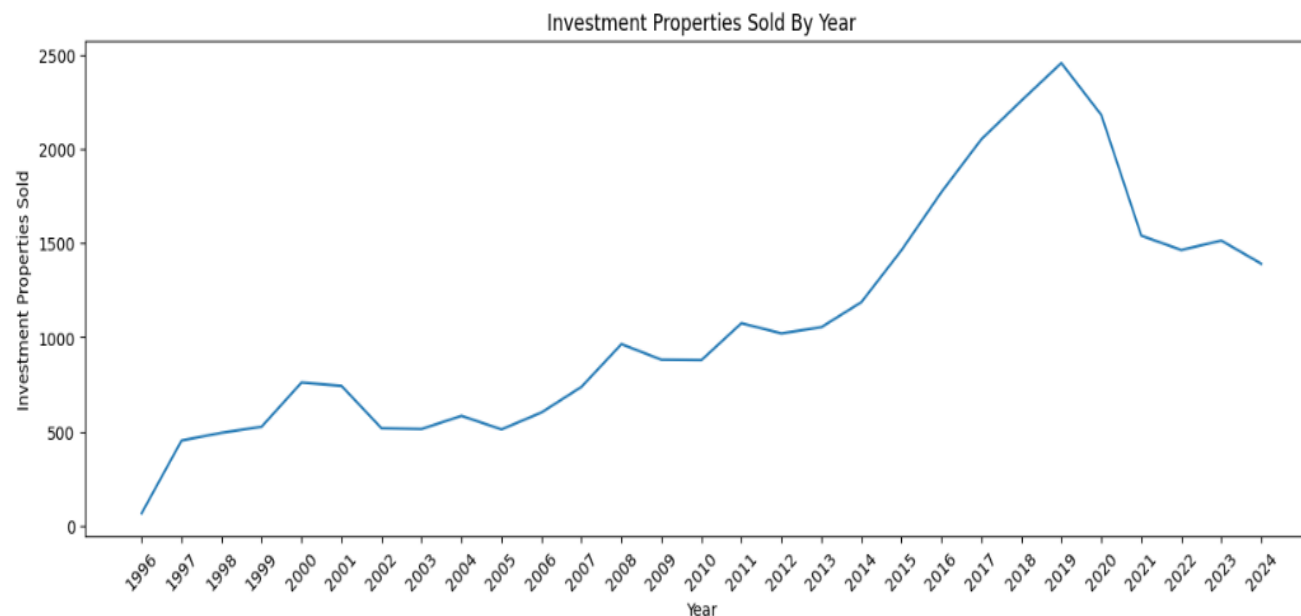
# Investment Properties Sold By Year

- As a real estate agent and an investor, I'm extremely interested in the amount of investing going on in the state
- The graph below show properties sold by a company for each year starting in 1996
- Based on the data in this MLS, I'm unable to capture the name of the person/entity who purchases the property; only who sells it.
- The entity was determined using Regex to match a pattern like LLC, Corporation, Management, etc
- Investment sales peaked in 2019 with almost 2500 homes being sold by corporations, but has since declines
- This peak occurs right before COVID and median sales prices exploding

```
1 query="""
2 SELECT year, COUNT(investment_sale) as investment_properties_sold
3 FROM gsmls_imputed_data
4 WHERE investment_sale = TRUE
5 GROUP BY year;
6 """
7
8 investment_data = pd.read_sql_query(query, tax_engine)
```

```
1 fig, ax = plt.subplots(figsize=(15,5))
2
3 # 2025 data excluded (year is not complete)
4 ax.plot('year', 'investment_properties_sold', data=investment_data[investment_data['year'] < 2025])
5 ax.set_title('Investment Properties Sold By Year')
6 ax.set_xlabel('Year')
7 ax.set_ylabel('Investment Properties Sold')
8
9 plt.xticks(range(1996, 2025, 1), rotation=45)
10 plt.show()
```

(1)



## Top Ten Municipalities for Investing in 2024 (Based on Investment Sales)

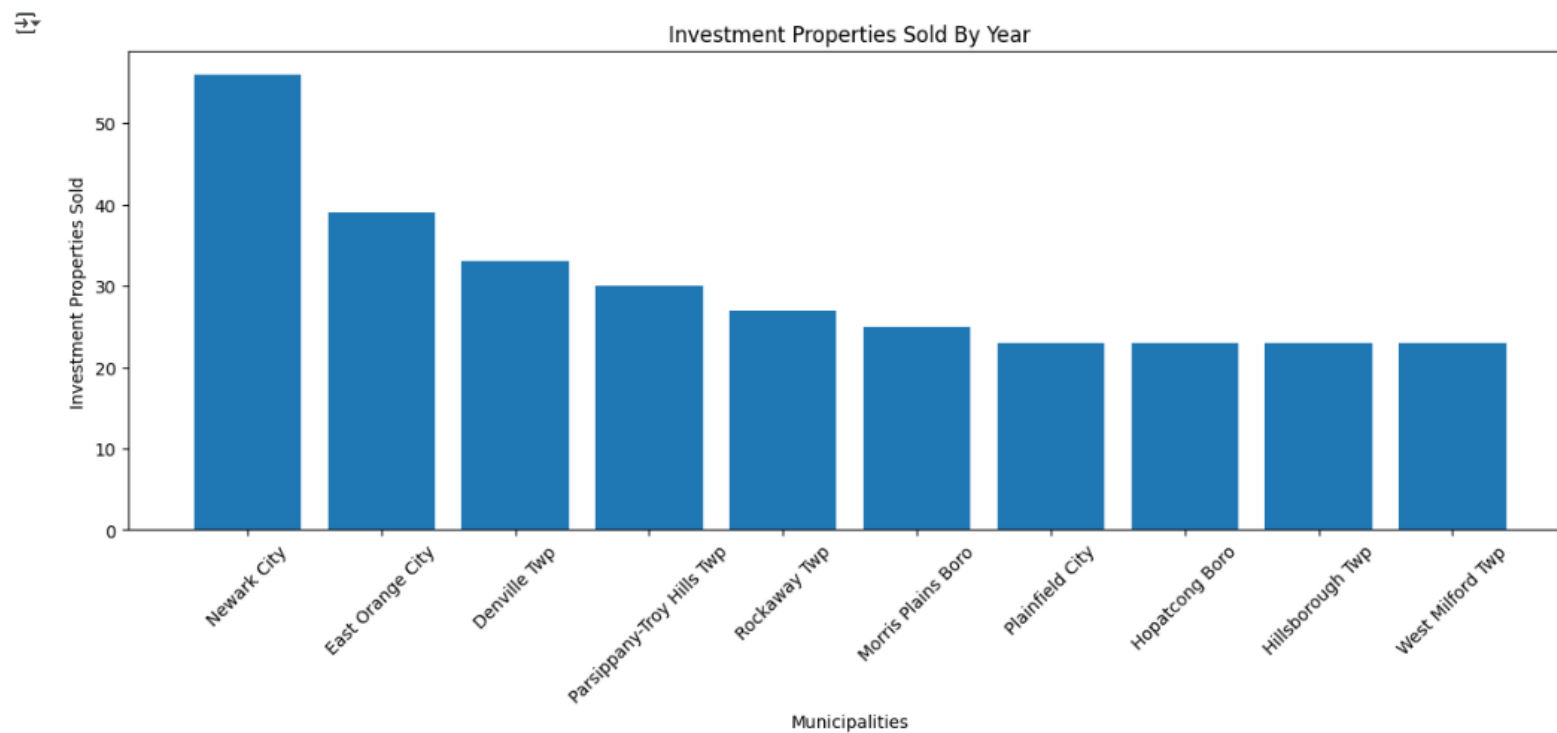
- The data shown here are the Top Ten Municipalities for Investing in 2024 (Based on Investment Sales)
- Six of the ten municipalities have a major NJ Transit line and multiple bus routes
- Through anecdotal research, these municipalities have been shown to be major destinations for residents migrating from NYC

```
1 query="""
2 SELECT town, county, year, COUNT(investment_sale) as investment_properties_sold
3 FROM gsmls_imputed_data
4 WHERE investment_sale = TRUE AND year = '2024'
5 GROUP BY town, county, year
6 ORDER BY investment_properties_sold DESC
7 LIMIT 10;;
8 """
9
10 investment_data = pd.read_sql_query(query, tax_engine)
```

1 investment\_data

|   | town                      | county   | year | investment_properties_sold |
|---|---------------------------|----------|------|----------------------------|
| 0 | Newark City               | Essex    | 2024 | 56                         |
| 1 | East Orange City          | Essex    | 2024 | 39                         |
| 2 | Denville Twp              | Morris   | 2024 | 33                         |
| 3 | Parsippany-Troy Hills Twp | Morris   | 2024 | 30                         |
| 4 | Rockaway Twp              | Morris   | 2024 | 27                         |
| 5 | Morris Plains Boro        | Morris   | 2024 | 25                         |
| 6 | Plainfield City           | Union    | 2024 | 23                         |
| 7 | Hopatcong Boro            | Sussex   | 2024 | 23                         |
| 8 | Hillsborough Twp          | Somerset | 2024 | 23                         |
| 9 | West Milford Twp          | Passaic  | 2024 | 23                         |





Top Ten  
Municipalities  
for Investing in  
2024 (cont'd)

# Top Ten Municipalities for Investing by Year (Based on Investment Sales)

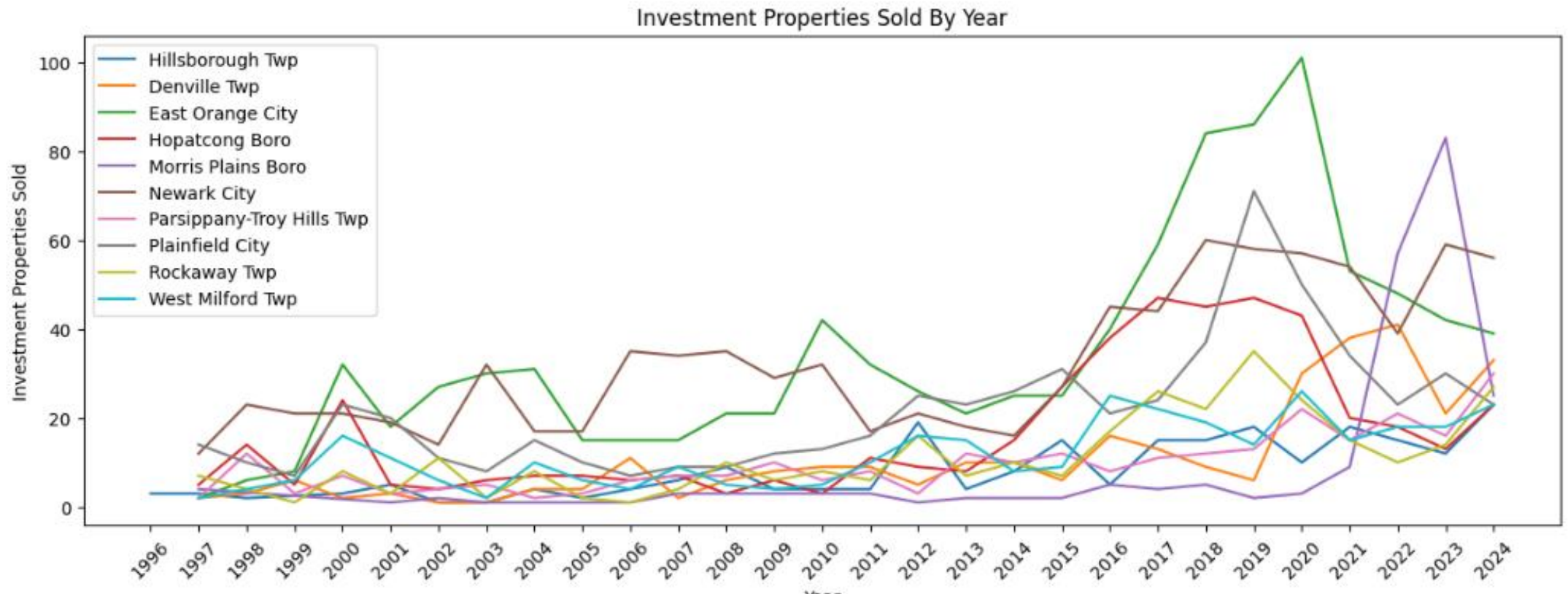
---

```
1 query="""
2 SELECT town, county, year, COUNT(investment_sale) as investment_properties_sold
3 FROM gsmls_imputed_data
4 WHERE investment_sale = TRUE AND town IN ('Newark City', 'East Orange City',
5 'Denville Twp', 'Parsippany-Troy Hills Twp', 'Rockaway Twp', 'Morris Plains Boro',
6 'Plainfield City', 'Hopatcong Boro', 'Hillsborough Twp', 'West Milford Twp')
7 AND county IN ('Essex', 'Morris', 'Union', 'Sussex', 'Somerset', 'Passaic')
8 GROUP BY town, county, year
9 ORDER BY year;
10 """
11
12 investment_data = pd.read_sql_query(query, tax_engine)
```

```
1 fig, axs = plt.subplots(figsize=(15,5))
2
3 for i in investment_data['town'].unique():
4
5     axs.plot('year', 'investment_properties_sold', data=investment_data[investment_data['town'] == i], label=i)
6
7 axs.set_title('Investment Properties Sold By Year')
8 axs.set_xlabel('Year')
9 axs.set_ylabel('Investment Properties Sold')
10
11 plt.xticks(range(1996, 2026, 1), rotation=45)
12 plt.legend()
13 plt.show()
```

## Top Ten Municipalities for Investing by Year (Based on Investment Sales)

- Most of the Top Ten Municipalities has their investment sales peaks between 2017 – 2020 with East Orange have the highest peak of them all in 2020
- Morris Plains had a later period of heavy investing occurring from 2021 and peaking in 2023
- The city of Newark has had two peak investment period occurring in 2018 and 2023 respectively





# Investment Properties Sold vs Municipal Median Sales Price (2024)

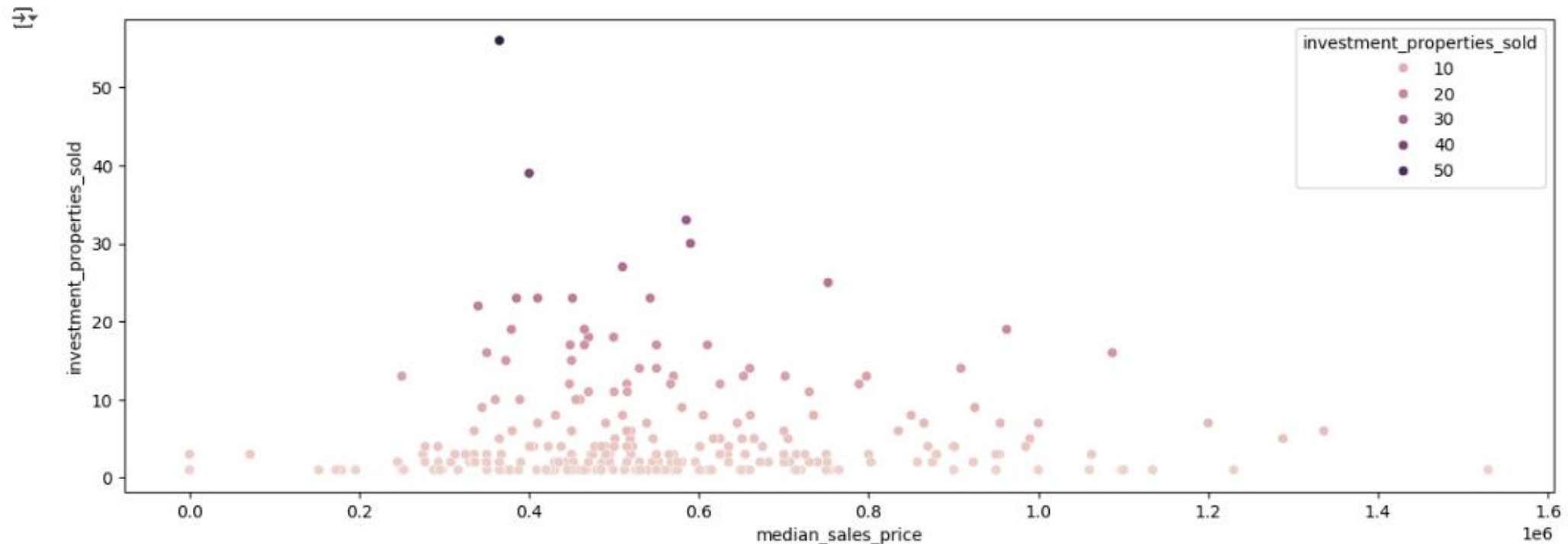
---

```
1 query="""
2 SELECT gsmls_imputed_data.town, gsmls_imputed_data.county, gsmls_imputed_data.year, percentile_cont(0.5) WITHIN GROUP(ORDER BY CAST(salesprice as numeric)) as median_sales_price, investment_table.investment_properties_sold
3 FROM gsmls_imputed_data
4 LEFT JOIN(
5   SELECT town, county, year, COUNT(investment_sale) as investment_properties_sold
6   FROM gsmls_imputed_data
7   WHERE investment_sale = TRUE and year = '2024'
8   GROUP BY town, county, year
9 ) as investment_table
10 ON investment_table.town = gsmls_imputed_data.town AND investment_table.county = gsmls_imputed_data.county AND investment_table.year = gsmls_imputed_data.year
11 WHERE gsmls_imputed_data.year = '2024'
12 GROUP BY gsmls_imputed_data.town, gsmls_imputed_data.county, gsmls_imputed_data.year, investment_table.investment_properties_sold;
13 """
14
15 investment_data = pd.read_sql_query(query, tax_engine)

1 fig, axs = plt.subplots(figsize=(15,5))
2
3 sns.scatterplot(data=investment_data, x='median_sales_price', y='investment_properties_sold', hue='investment_properties_sold')
4 plt.show()
```

## Investment Properties Sold vs Municipal Median Sales Price (2024)

- I wanted to see what the distribution was like for total investments in municipalities across the full range of median sales prices (Is investing targeted to primarily to lower prices areas?)
- It seems like investing is concentrated in municipalities with median sales prices ranging from approximately \$300k to \$800k
- This range is a lot higher than my assumed range of approximately \$200k to \$500k

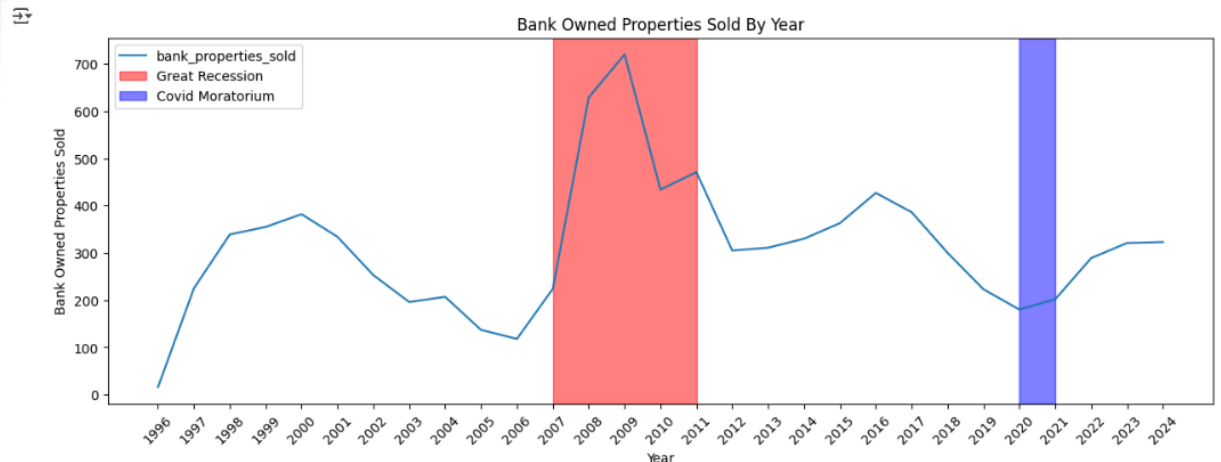


# Bank Owned Properties By Year

- The graph below shows different periods of the real estate market where foreclosures were on the rise
- As expected, the highest rise and peak of properties foreclosed and sold by the bank occurred during the Great Financial Crisis
- Highlighted in blue was the period of COVID where unemployment was at it's highest. However, a national mortgage moratorium was passed where homes could not be seized due to late/delinquent mortgage payments
- Bank foreclosures have steadily been on the rise and have plateaued from 2023-2024

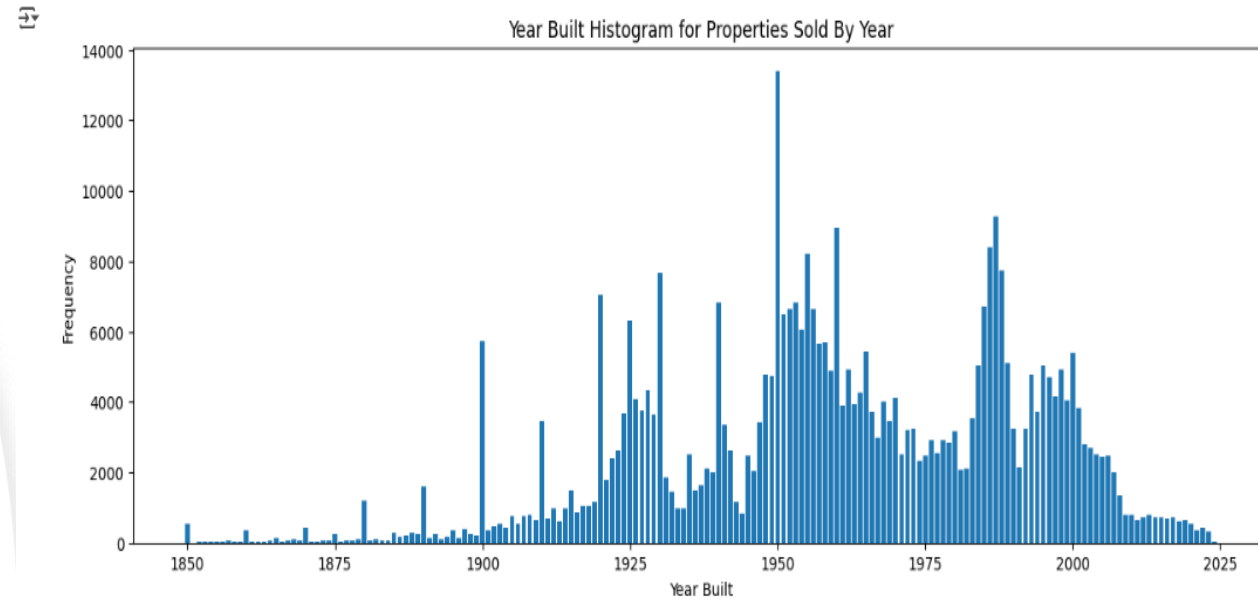
```
1 query="""
2 SELECT year, COUNT(bank_owned) as bank_properties_sold
3 FROM gsmls_imputed_data
4 WHERE bank_owned = TRUE
5 GROUP BY year;
6 """
7
8 bank_owned_data = pd.read_sql_query(query, tax_engine)
```

```
1 fig, axs = plt.subplots(figsize=(15,5))
2
3 axs.plot('year', 'bank_properties_sold', data=bank_owned_data[bank_owned_data['year'] < 2025])
4 axs.set_title('Bank Owned Properties Sold By Year')
5 axs.set_xlabel('Year')
6 axs.set_ylabel('Bank Owned Properties Sold')
7
8 plt.axvspan(2007, 2011, color='red', alpha=0.5, label='Great Recession')
9 plt.axvspan(2020, 2021, color='blue', alpha=0.5, label='Covid Moratorium')
10 plt.xticks(range(1996, 2025, 1), rotation=45)
11 plt.legend()
12 plt.show()
```



# Distribution of Homes Sold by the Year Built

- Homes built in the 1950s and 1980s dominate the existing sold market
- New construction Single family homes have seen a heavy decline since the early 2000s
- Possible factors for the decline:
  - Lack of available land to develop
  - Lengthy process in which new construction projects need to be approved
  - Rise in the cost of materials
  - New construction and existing homes have become less affordable



```
1 query="""
2 SELECT year_built, COUNT(year_built) as year_built_count
3 FROM gsmls_imputed_data
4 WHERE status_short = 'SD'
5 GROUP BY year_built
6 ORDER BY year_built;
7 """
8
9 year_built_data = pd.read_sql_query(query, tax_engine)
10 year_built_data = year_built_data[(year_built_data['year_built'] >= 1850) & (year_built_data['year_built'] < 2025)]
```

```
1 fig, axs = plt.subplots(figsize=(15,5))
2
3 axs.bar('year_built', 'year_built_count', data=year_built_data)
4 axs.set_title('Year Built Histogram for Properties Sold By Year')
5 axs.set_xlabel('Year Built')
6 axs.set_ylabel('Frequency')
7
8 plt.show()
```

# Types of Flooring Observed in Homes Sold (2024)

```
1 [i for i in test_data.columns if '_FLOORS' in i]
```

```
['WOOD_FLOORS',  
'MARBLE_FLOORS',  
'TILE_FLOORS',  
'CARPET_FLOORS',  
'VINYL_FLOORS',  
'LAMINATE_FLOORS',  
'STONE_FLOORS',  
'PARQUET_FLOORS']
```

✓ Create a pie chart displaying the percentage of flooring types used in all homes currently sold to date

```
[ ] 1 # Iterate over the list of flooring types to create a new table with all flooring types  
2 #used in sold properties  
3  
4 data_list = []  
5  
6 for i in [i[:-7] for i in test_data.columns if '_FLOORS' in i]:  
7  
8     query=f"""  
9     SELECT \"YEAR\", COUNT(\"{i}_FLOORS\") as {i.lower()}  
10    FROM res_properties  
11   WHERE \"STATUS_SHORT\" = 'SD' AND \"{i}_FLOORS\" IS TRUE  
12   GROUP BY \"YEAR\"  
13   ORDER BY \"YEAR\"  
14   """  
15  
16 floor_data = pd.read_sql_query(query, gsmls_engine)  
17 # Calculate the percentage change in the use of flooring compared to the prior year  
18 floor_data[f'{i.lower()}_pct_change'] = (floor_data[f'{i.lower()}'].pct_change() * 100).round(2)  
19  
20 data_list.append(floor_data)
```

```
1 # Use the dataframe in index 0 to merge all dataframes to  
2  
3 total_data = data_list[0]  
4  
5 for i in data_list[1:]:  
6  
7     total_data = total_data.merge(i, on='YEAR')
```

```
1 # Remove the percentage change columns and transpose the dataframe  
2 # to make the columns as years and the index the flooring types  
3 new_df = total_data[[i for i in total_data.columns if 'pct' not in i]]  
4 new_df = new_df.T
```

```
1 # Drop the row which represents the year  
2 new_df.drop(new_df.index[0], inplace=True)
```

```
1 fig, ax = plt.subplots(figsize=(10, 10))  
2 ax.pie(new_df[16], labels=new_df.index, autopct='%1.1f%%')  
3 plt.show()
```

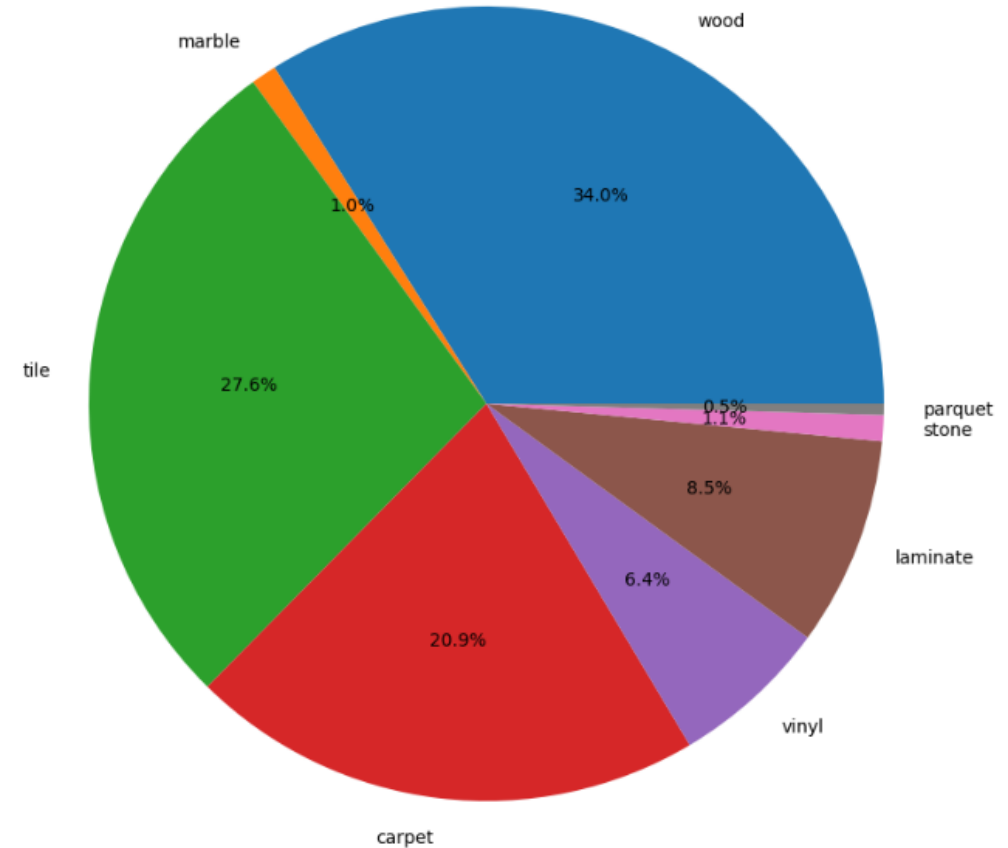


# Types of Flooring Observed in Homes Sold (2024) cont'd

- Types of flooring used in a property can add value, especially if one is considering selling in the future
- The most common floor types are wood, tile and carpet, respectively, with most properties having at least two of the three present
- Parquet, marble and stone are the least common flooring types. These flooring types are found in higher priced properties



Flooring Types Used in Homes Currently Sold (2024)





# Section 3: Data Visualization (Folium Dashboard)



### NJ Comps Dashboard

Previously run comps

No options to select

Choose County

Choose an option

☐ Has the listing price been reduced/increased since being listed?

Reset Get Comps

Required Information

Address ?

123 Main Street

Municipality

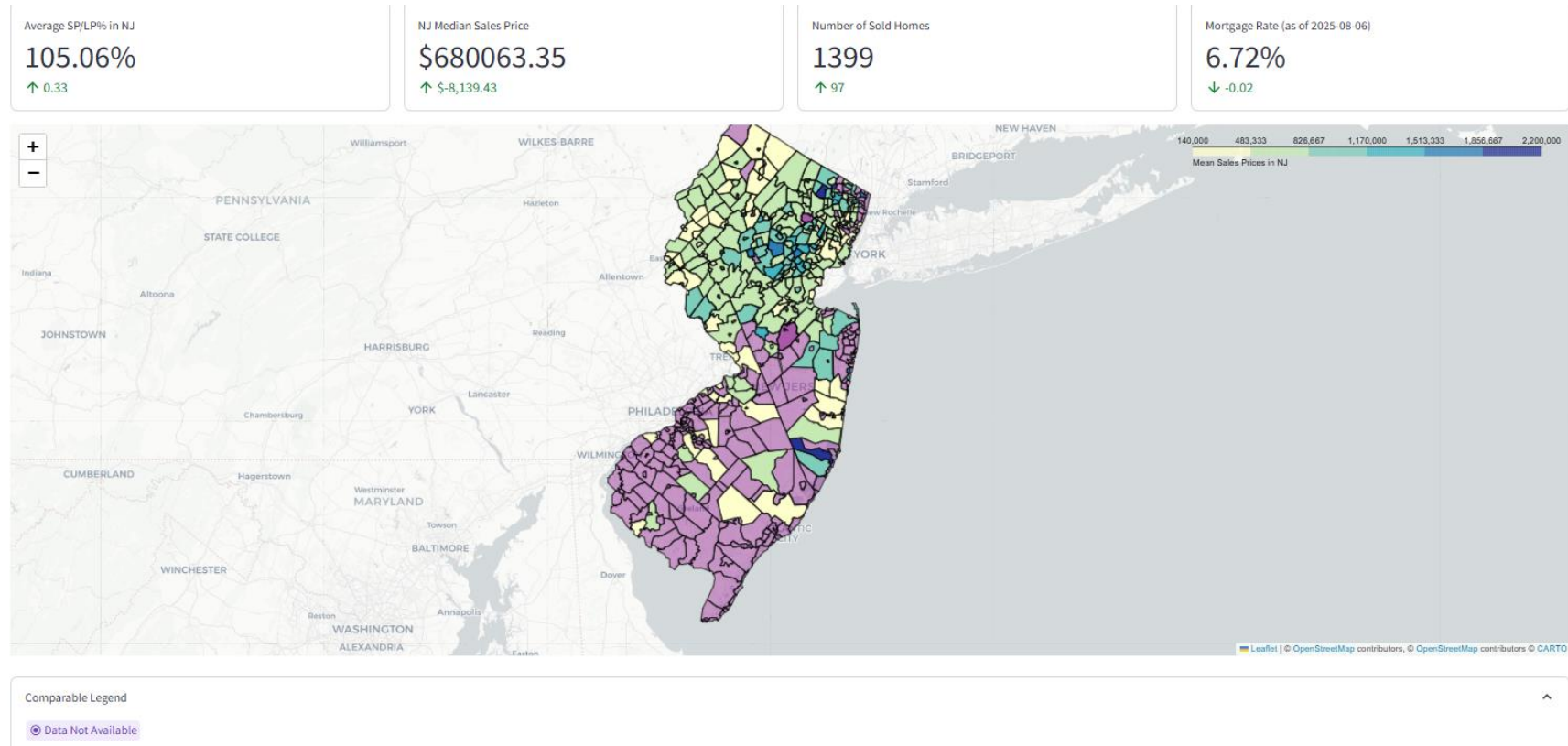
No options to select

Zipcode

00000

The widget with key "listdate" was created with a default value but also had its value set via the Session State API.

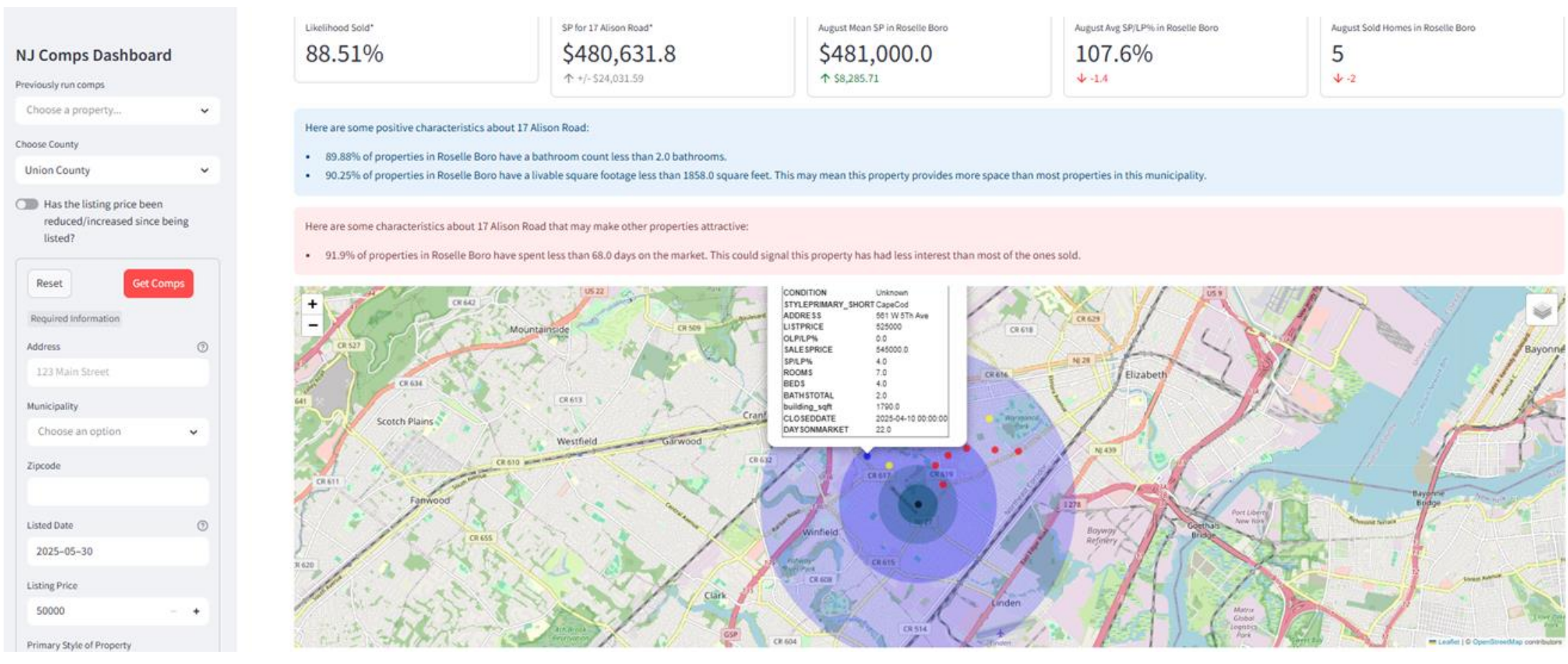
Listed Date ?



## NJ Comparables Dashboard

- The NJ Comparables Dashboard was designed to be a one-stop shop for real estate agents, home buyers and home sellers alike in the state of NJ to generate quick comparable properties as well as gauge a subject property's sales price based on its physical and locational characteristics.
- In addition, one can assess local real estate market activity with closed homes (metrics such as homes available, currently under contract, withdrawn/expired coming soon) to help recognize what transactions are occurring on a weekly basis.





## Comparables Results

- This screen shows the comparable results to the target house
- The larger blue circles on the map depict the radius from the target property in which a comp would be considered (0.25 – 2 miles)
- The quality of the comps are color coded to show which ones should be shown the most focus with blue meaning the best comp and red being not-so significant (legend appears on the dashboard)
- A regression deep neural network is deployed to help predict the price which the target property can sell with a +/- 5.1% average MOE
- The dashboard provides interesting statistical information about the target property's intrinsic characteristics if they stand out against the population



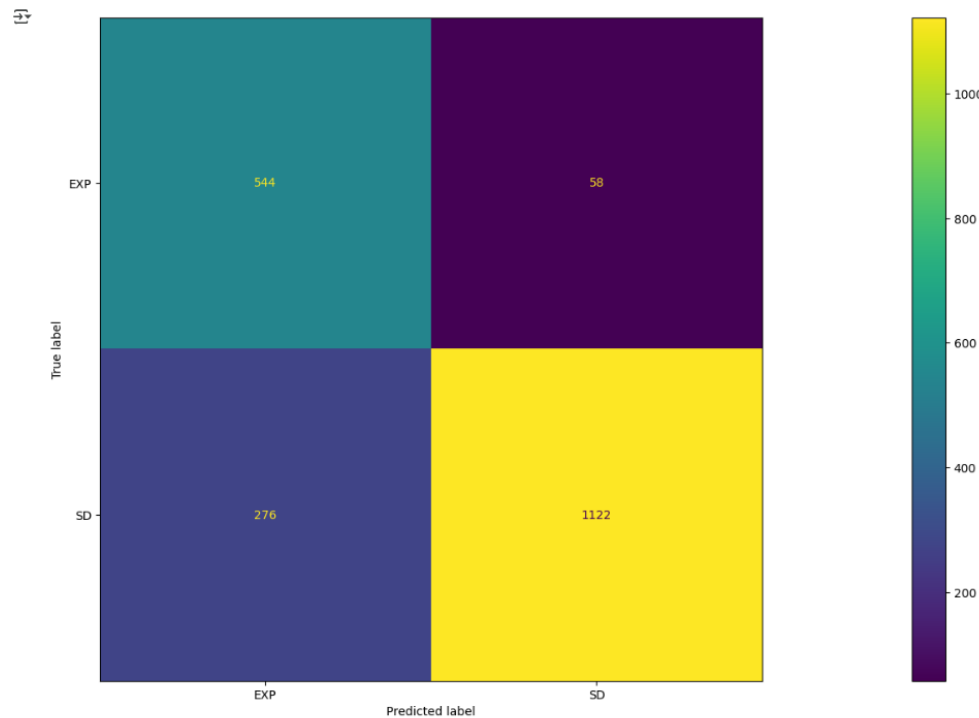
# Section 5: Predictive Analysis (Classification & Regression)





# Classification Accuracy & Confusion Matrix

- The classification model seems to perform well for predicting 'Sold' homes, but needs some improvement for determining when a home will expire on market
- Recall for true 'EXP' labeled instances are very good
- The dataset is heavily weighted towards homes sold; retraining with balanced training data will be necessary



|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| EXP          | 0.66      | 0.90   | 0.77     | 602     |
| SD           | 0.95      | 0.80   | 0.87     | 1398    |
| accuracy     |           |        | 0.83     | 2000    |
| macro avg    | 0.81      | 0.85   | 0.82     | 2000    |
| weighted avg | 0.86      | 0.83   | 0.84     | 2000    |

# Regression Model Evaluation

| Metrics                                                              | Benchmark                                                                                                                                 | Model Results             |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Median Sales Price                                                   | ---                                                                                                                                       | \$600,000                 |
| R2 Score                                                             | 0.95 or greater                                                                                                                           | 0.968                     |
| Mean Absolute Error (MAE)                                            | A value between 5-10% of the median target value is considered good                                                                       | \$ 39,434.81              |
| Root Mean Squared Error<br><br>(Normalized RMSE) <a href="#">(1)</a> | A value between 5-10% of the median target value is considered good<br><br>Range: 0 – 1.<br>Values closure to 0 represents a better model | \$70,555.21<br><br>(0.17) |
| Mean Absolute Percentage Error (MAPE) <a href="#">(2)</a>            | A value between 5-20% of the median target value is considered good                                                                       | 5.27%                     |

- Coefficient of determination meet the criteria for a good predictor. This metric will be monitored as new data is acquired
- The MAE and RMSE are **6.57%** and **11.76%** of the median target value, respectively. The MAE meets the benchmark criteria, however, the RMSE still shows outliers still present an issue.
- The MAPE meets the benchmark criteria and shows that the model can be relied upon for its predictions

# Conclusions

---

- The regression DNN captures the 96.8% of the sales price variance from the available data
- The classification DNN has an F1 Score of 85%
- More training data needs to be acquired to improve both models, but especially for the classification. There are still some features needed to help distinguish between a property which is sold/not sold
- Collected data is concentrated in the Northern and Central municipalities. Access to data for South Jersey is necessary for better model generalization and improvement
- Home prices continue to rise YoY while total numbers of home sales continue to decline
- The median days a property spend on the market before being sold hasn't been above 30 days since 2021. This signifies heavy buyer demand for homes.
- Bank foreclosures have steadily increased since the COVID mortgage moratorium ended in 2021 but has plateaued in 2023 - 2024
- Properties bought by investors in NJ peaked in 2019 and have declined since
- Homes built in the 1950s and 1980s dominate the existing homes market. There has been a lack of new construction homes built since the early 2000s

# Next Steps

---

- Train a multi-classification CNN to label real estate images for the property's quality/condition and the area/section of the house
- Change the “Unknown” value of a property's “Condition” to the predicted classification label
- Retrain the classification network now using “Condition” as a feature
- Use sentiment analysis on a property's listing remarks also understand the condition of the house
  - Do an analysis of the results of the visual prediction vs the sentiment prediction. Which one more accurately depicts the quality of the house? Can I use the sentiment prediction as a regularization (of some sort) for the image quality prediction?
- Create dashboard to monitor the results of the neural networks and keep track of each iteration and change log of the retraining/updates

# Appendix

---

- (1) <https://www.statology.org/what-is-a-good-rmse/>
- (2) <https://www.statology.org/how-to-interpret-mape/>





**Thank You**